Java Practice Questions — REVISION

Topics covered: Variables, Data Types, Operators, If-Else, Class, Objects, Methods, Static Keyword, Loops, Arrays

Question 1: (EASY)

Scenario: A local cinema wants a simple program to handle ticket bookings for movies. Every ticket has a movie name, a seat type (`"Regular"` or `"Premium"`), and a base price. Premium seats cost 1.5 times the base price. The cinema also wants to keep a running count of how many tickets have been sold in total, across *all* bookings — not per ticket.

Requirements:

1. Create a class `Ticket` with instance variables: `movieName` (String), `seatType` (String), `basePrice` (double).
2. Add a **static** variable to track the total number of tickets sold across all `Ticket` objects.
3. Add a method `calculatePrice()` that returns the final price: if `seatType` is `"Premium"`, multiply `basePrice` by 1.5; otherwise just return `basePrice`.
4. Add a method `bookTicket()` that increases the static counter by 1 and prints the booking details (movie name, seat type, and final price).
5. In `main`, create **two** `Ticket` objects with different movies/seat types using the dot operator to set their fields, call `bookTicket()` on each, then print the total tickets sold so far.

Sample Input (hardcode these values in your objects):

Movie	Seat Type	Base Price
Avengers	Premium	200.0
Inside Out 2	Regular	150.0

Sample Output:

```
Ticket booked for: Avengers
Seat Type: Premium
Price to pay: Rs. 300.0
Ticket booked for: Inside Out 2
Seat Type: Regular
Price to pay: Rs. 150.0
Total tickets sold so far: 2
```

Question 2: (Medium)

Scenario: A school wants a program that generates a report card for each student. Every student has a name and marks in 5 subjects (out of 100 each). The school also wants to know how many report cards have been generated in total during the program's run.

Requirements:

1. Create a class `ReportCard` with instance variables: `studentName` (String) and `marks` — an array of 5 integers.
2. Add a **static** variable to count how many report cards have been generated overall.
3. Add a method `calculateTotal()` that **loops** through the `marks` array and returns the sum.
4. Add a method `calculateAverage()` that returns the total divided by 5 (as a double).
5. Add a method `getGrade()` that returns a grade based on the average using if-else:
   - average ≥ 90 → `"A"`
   - average ≥ 75 → `"B"`
   - average ≥ 60 → `"C"`
   - else → `"D"`
6. Add a method `generateReportCard()` that increases the static counter and prints: student name, total marks, average, grade, followed by a separator line `------------------------`.
7. In `main`, create two `ReportCard` objects with the marks below, call `generateReportCard()` on each, then print the total number of report cards generated.

Sample Input:

Student	Marks
Asha	85, 92, 78, 88, 95
Bibek	55, 60, 48, 65, 50

Sample Output:

```
Student: Asha
Total Marks: 438
Average: 87.6
Grade: B
------------------------
Student: Bibek
Total Marks: 278
Average: 55.6
Grade: D
------------------------
Total report cards generated: 2
```

Question 3: (Hard)

Scenario: A community bank wants a simple simulator for a customer's account. The bank's name is shared by every account (it doesn't change per object), and the bank wants to track the total number of accounts opened across the whole program. A customer can deposit and withdraw money, and the bank wants to process a whole batch of transactions for an account in one go — where a positive number means a deposit, and a negative number means a withdrawal.

Requirements:

1. Create a class `BankAccount` with instance variables: `accountHolder` (String) and `balance` (double).
2. Add **static** variables: `bankName` (String, set to `"Nepal Community Bank"`) and `totalAccountsOpened` (int).
3. Add a method `openAccount(String name, double initialDeposit)` that sets the account holder and balance, increases `totalAccountsOpened`, and prints a welcome message with the bank name and the initial balance.
4. Add a method `deposit(double amount)` that adds the amount to balance and prints the deposited amount and new balance.
5. Add a method `withdraw(double amount)` that, using if-else, checks if the amount is less than or equal to the balance:
    - If yes: subtract it and print the withdrawn amount and new balance.
    - If no: print a failure message saying the withdrawal failed due to insufficient balance.
6. Add a method `processTransactions(double[] transactions)` that **loops** through an array of transaction amounts. For each value: if it's zero or positive, call `deposit()`; if it's negative, call `withdraw()` with the positive version of that amount. After the loop, print the final balance for that account holder.

7. In `main`, open one account for "Priya" with an initial deposit of 1000.0, then process this batch of transactions: `{500.0, -200.0, -2000.0, 300.0}`. Finally, print the bank name and total accounts opened using the class name (not an object).

Sample Output:

```
Welcome to Nepal Community Bank, Priya!
Initial Balance: Rs. 1000.0
Deposited Rs. 500.0 | New Balance: Rs. 1500.0
Withdrew Rs. 200.0 | New Balance: Rs. 1300.0
Withdrawal of Rs. 2000.0 failed: Insufficient balance
Deposited Rs. 300.0 | New Balance: Rs. 1600.0
Final Balance for Priya: Rs. 1600.0
Total accounts opened at Nepal Community Bank: 1
```